Not approved alternative concept. This PR/FAQ explores a product direction for Lux as a self-extending display server. It is not the canonical Lux architecture, is not an approved roadmap, and is not under active development. The ideas here may inform future work. See also: `concept-self-extending-display.md` (vision) and `concept-extension-architecture.tex` (technical design).

Self-Extending Display Server Gives Terminal-Hosted AI Agents a Visual Surface Without Giving Up Local Compute

Agents author new visual capabilities in minutes — native GPU-accelerated rendering via ImGui and SDL2, no web stack, no vendor compute required

Open-source display server with community extension registry, three rendering modes, and an in-display agent SDK

Press Release

AUSTIN, TX — Punt Labs today released Lux, a GPU-accelerated display server that gives terminal-hosted AI coding agents a persistent, shared visual surface — without moving compute to a vendor's cloud. The most powerful agentic coding tools (Claude Code[1], Codex) run in the terminal on the developer's own machine; Lux gives them a native visual surface built on Dear ImGui[2] and SDL2, keeping everything local. Unlike vendor dashboards that require running the entire agent on remote compute, or terminal scrollback that has no structure at all, Lux extends itself: when an agent needs a visualization that does not exist, it authors a new extension, installs it, and renders — in under five minutes, without a product release. Extensions are shareable through a community registry with ratings and verification badges, so most users never need to author code themselves (see FAQ 6).

Problem

Developers running AI coding agents coordinate multiple agents across repos, sessions, and machines. The most capable agentic tools — Claude Code[1], Codex — are terminal-hosted, running on the developer's own machine with full tool use, MCP integration, and multi-turn autonomous operation. As Steve Yegge has noted, coordinating multiple agents is "hard and exhausting"[3]. When an agent produces structured output — a table, a chart, an architecture diagram — the developer sees raw text or takes an ad hoc screenshot.

Vendors are responding with web dashboards to manage these agents. But those dashboards often require running the *entire agent* on the vendor's compute — not just the LLM call, the agent itself. The developer trades local control for visual management (see FAQ 2). IDE-hosted tools like Cursor offer agent management features but remain all-text; Claude Desktop produces novel visual interfaces but lacks agentic power — no tool use, no MCP, no multi-turn autonomous work. The result: the most powerful agents have no visual surface, and the tools with visual surfaces lack agentic power.

Solution

Lux is a persistent local service — like Ollama[4] for inference, but for display. Agents connect via an HTTP service API and send structured content. The display renders it in a native GPU-accelerated window at 60 fps using Dear ImGui[2] and SDL2. Multiple agents share one display, each in its own frame, with clear attribution. Everything runs on the developer's machine — no vendor compute, no cloud dependency.

The core handles transport, protocol dispatch, extension registry, frame management, render loop, and safety. Everything the user sees — every element kind, every applet, every theme — is an extension. Lux ships with a starter pack of bundled extensions (text, tables, plots, diagrams, interactive controls), but the vocabulary is not fixed.

When no installed extension covers a visualization need, an agent can author one: generate the manifest and renderer code, run tests, and install — all through the service API. The user approves via a consent dialog. The extension persists in the local registry and is available for every future session. Publishing to the community registry lets anyone install it with one command (see FAQ 19).

Lux is not a passive display. The bidirectional write path — user and agent both driving the display, the display driving agents back — was the key lesson from v1 (see FAQ 18). An in-display agent built on the Anthropic Python SDK can receive intent from external agents, author extensions, introspect frames, and take screenshots for iterative GUI refinement — a technique proven during the Postern/Pharo work[5, 6] (see FAQ 14).

Customer Quote

> *"I run six Claude Code sessions across three repos. Before Lux, I had six terminal windows and no idea what any of them were doing unless I scrolled back. Now I glance at one window and see beads boards, test dashboards, and architecture diagrams — one frame per agent, all live. Last week an agent needed a dependency graph and there was no built-in widget for it. It wrote one. Took three minutes. I've been using that widget every day since."*
>
> — **Maya Torres**, Staff Engineer at a Series B DevTools Startup

Getting Started

Getting started takes one command:

1. Install:
   `curl -fsSL https://raw.githubusercontent.com/punt-labs/lux/main/install.sh | sh`
2. Restart Claude Code. The Lux display opens automatically when any agent sends visual output.
3. Browse community extensions: `lux ext search <keyword>` and install with `lux ext install <name>`.

No API key is required for the display or community extensions. The optional in-display agent (for authoring new extensions) requires an Anthropic API key.

Spokesperson Quote

> "Lux proved the display has value — the beads browser alone changed how we work. Pharo proved a self-extending system is valuable — an agent that writes its own UI code, tests

> it, and persists it. The self-extending display concept combines both: the GPU-accelerated ImGui/SDL2 display, plus the self-extension model from Pharo. Terminal agents are the most powerful agentic tools available, but they have no visual surface. Web dashboards provide visuals but pull your agent onto vendor compute. Lux fills that gap — native rendering, local-first, and the result is that 'Lux can't do X' stops being a limitation and becomes a five-minute problem."
>
> — **Jim Freeman**, Punt Labs

Call to Action

Lux is free and open-source under the MIT license. Visit `https://github.com/punt-labs/lux` to install, browse the extension registry, or contribute. The community registry is open for submissions.

Frequently Asked Questions

External FAQs

Q1: What is Lux and who is it for?

Lux is a GPU-accelerated display server — a persistent local visual surface that AI coding agents write to and read from, built on Dear ImGui[2] and SDL2. It is for developers who run terminal-hosted coding agents (Claude Code, Codex, or similar) and need structured visual output — tables, charts, diagrams, interactive controls, and custom visualizations — without moving their agents to vendor compute. Lux runs on macOS and Linux.

Q2: How is this different from terminal scrollback, Claude Desktop, or vendor dashboards?

Terminal agents (Claude Code, Codex) are the most powerful but have no visual output. Claude Desktop has visual output but lacks agentic power (no tool use, no MCP). Vendor web dashboards provide visuals but require running the entire agent on vendor compute. IDEs like Cursor manage agents but output is text-only.
Lux fills the gap: a GPU-accelerated visual surface for terminal-hosted agents, running on the developer's machine. When you need a visualization that does not exist, an agent authors it in minutes and it persists. See FAQ 10 for detailed competitive analysis.

Q3: How do I get started?

One curl command installs Lux and registers the Claude Code plugin. Restart Claude Code. The display opens automatically when any agent sends visual output. Most users see their first visualization within 60 seconds of install.

Q4: How much does it cost?

Free and open-source (MIT). Community extensions are free. The optional in-display agent for authoring new extensions requires an Anthropic API key (usage-based, same pricing as any Claude API call).

Q5: What happens to my data?

All data stays local. Lux communicates over a Unix domain socket (localhost only by default) or TCP with TLS for remote agents. No telemetry, no data leaves your machine. Extensions run in-process; the consent model gates what code executes.

Q6: Can I use Lux without enabling LLM-powered extension authoring?

Yes. The community registry provides pre-built extensions authored, tested, and rated by other users. Install with `lux ext install <name>`. No API key required. No LLM runs in your Lux instance. The extension store shows verification badges — lint passing, CI green, test coverage, community ratings — so you can assess quality before installing.
A small number of extension authors produce extensions consumed by a larger population of users who never need to write code.

Q7: What Python visualization libraries can Lux host?

Lux supports three rendering modes. **ImGui mode**: extensions render natively in the Lux window using Dear ImGui — full interactivity, single-window UX. **Texture mode**: extensions produce pixel buffers (matplotlib, PIL, plotly snapshots) displayed as images in the Lux window — limited interactivity but zero integration effort. **Window mode**: extensions own their own OS window (Qt, tkinter, Jupyter kernels) — full interactivity, coordinated lifecycle, separate window.

Any Python library that can render to a buffer or open a window works.

Internal FAQs

Value & Market

Q8: What is the total addressable market?

The primary market is developers using terminal-hosted AI coding agents. Claude Code and Codex are the leading tools — the most capable agentic solutions, both terminal-hosted. Anthropic reports millions of Claude Code users. The secondary market is any developer running Python-based AI pipelines who needs visual output without cloud dependency.
Bottoms-up: 1% adoption of active Claude Code users (free, zero-config install) is tens of thousands of active installations. Registry value scales with adoption — more users means more extensions, which means more users. [CITATION NEEDED: Claude Code user count]

Q9: What evidence do we have that customers want this?

Lux shipped in March 2026 and has been in daily use across Punt Labs projects since. The beads issue browser — a filterable table that auto-refreshes on every `bd` command — runs in every active agent session. The current product proved three things: (1) the display has value, (2) the hook architecture (PostToolUse auto-refresh) makes it feel live, and (3) agents and humans share one display surface effectively.
The self-extension hypothesis is validated by the Postern/Pharo work[5]: 199 classes, 2,358 methods, and 765 tests written entirely by agents driving a live image via HTTP — authored UI code, screenshotted, iterated, committed, all without human GUI interaction.
What has *not* been proven: whether community sharing works, whether the three rendering modes compose well, and whether the in-display agent is useful outside Punt Labs. These are the hypotheses this concept must test.

Q10: Who are the competitors and why will we win?

The competitive landscape divides by two axes: agentic power and visual surface.

- **Claude Code / Codex** — Terminal-hosted, the most powerful agentic tools: full tool use, MCP, multi-turn autonomous work. No visual output surface beyond scrollback. *Lux's primary integration targets.*
- **Claude Desktop** — Produces novel visual interfaces conceptually similar to the self-extending display described here. But it lacks Claude Code's agentic power: no tool use, no MCP, no multi-turn autonomous operation. Cannot drive complex coding workflows.
- **Cursor** — IDE with agent management features, but all text — no visual output surface for structured data. Extensible via VS Code extensions, but these are IDE-scoped and written by hand.
- **Vendor web dashboards** — Provide visual management but often require running the *entire agent* on vendor compute — not just the LLM call, the agent itself. The developer gives up local control for visual output.
- **Jupyter notebooks** — Rich visual output via kernel/display protocol. Not agent-first; requires notebook UX.

Primary threat: Anthropic-native visual output. Anthropic has every incentive to build visual panels into Claude Code. A basic built-in display narrows Lux's install-base advantage. An extensible one narrows differentiation to "local GPU-accelerated vs. web-native." Survival strategies: (a) Lux's extension ecosystem reaches critical mass before Anthropic ships, becoming the de-facto standard; (b) Lux becomes a complementary extension provider for whatever

Anthropic builds; (c) native ImGui/SDL2 rendering and fully local operation remain valuable for developers who refuse to move agents to vendor compute.
Structural advantage: local-first + self-extending. Lux grows its vocabulary via LLM-authored extensions without any vendor release. Vendor dashboards ship features on their schedule and require their compute. Lux delivers a new capability in minutes, on the developer's machine. A competitor copying the extension concept would have to abandon web-only rendering to integrate Python's visualization libraries — a platform-level decision they are unlikely to reverse.

Q11: What is your next step to validate the concept?

Two parallel tracks.
External demand: Interview 5–10 developers running 3+ concurrent agent sessions daily (staff-level, startups or mid-size companies). Key questions: (a) how do you track what your agents are doing? (b) when an agent produces structured output, how do you consume it? (c) would you install a display tool if it took 60 seconds? Go/no-go: if fewer than half describe agent visibility as a top-3 pain point, revisit the value proposition before committing to later phases.
Technical: Build the extension loader and registry as the first milestone. Migrate existing element kinds to registered extensions. Ship 3–5 reference extensions (matplotlib plot, terminal emulator, sankey diagram). Measure: can an agent author and install a new extension in under five minutes? Run the loop 10 times and report median and range.

Technical

Q12: What are the major technical risks?

1. **Extension safety.** Python has no in-process sandbox; extensions run with full process access. Mitigation: exception isolation (crash → error placeholder, core keeps rendering), consent dialog on install, community verification badges (lint, CI, tests). The trust model is identical to `pip install` — the user is the security boundary.
2. **Python dynamism limits.** Python cannot do Smalltalk's `become` or image persistence. Extensions persist as files; the process is disposable. Adequate for hot-loading new element kinds, but the display cannot save arbitrary runtime state across restarts.
3. **Three rendering modes composing well.** ImGui, texture, and window modes coexist in one session. The risk is UX fragmentation — separate OS windows feel disconnected. Mitigation: prefer ImGui mode; texture mode for most Python viz libraries; window mode as escape hatch only.
4. **Community registry bootstrapping.** A registry with no extensions has no users. Mitigation: ship 20+ starter-pack extensions plus 5+ reference extensions from day one.

Q13: What is the estimated development timeline?

Phased delivery, ordered by dependency. Scope compresses by cutting later phases.

1. **Extension system.** Registry, loader, manifest format, safety wrapper. Migrate existing element kinds to extensions.
2. **Service API.** HTTP endpoints (ollama-shaped). MCP adapter becomes a thin translation layer. Auto-documenting `/help` endpoint.
3. **Reference extensions.** Matplotlib, terminal (pyte), plotly snapshot, custom ImGui widgets. Proves the extension contract with real libraries.
4. **In-display agent.** Anthropic SDK integration. LuxExchange tool-use loop. Leaf and composite commands. Permission policy. Screenshot-driven iteration.

5. **Community registry.** GitHub-hosted. CLI: `lux ext install/publish/search`. Verification badges. Ratings.
6. **Remote transport.** TCP with TLS. Multi-machine agent sessions.

Cut line: phases 5–6 are valuable but not launch-blocking. Core value is in phases 1–3.

Q14: What is the relationship to Pharo and Postern?

Complementary, not competing. Pharo is the ambitious live-environment substrate: full image persistence, introspection, Morphic rendering. Postern[5] exposes Pharo over HTTP. The Claude Agent SDK for Smalltalk[6] provides the in-image tool runner.

Lux is the pragmatic local-first path. It shares the self-extension philosophy — agent authors code, code persists, system grows — but uses Python's ecosystem instead of Smalltalk's.

The two connect: a Lux window-mode extension can host Pharo-rendered content via Postern, and extensions designed in Morphic can be exported as Lux extensions for Python consumers.

Business

Q15: What is the revenue model?

Lux is free and open-source (MIT). Follow-on revenue opportunities: hosted managed instances, premium verified extensions, enterprise support. None are launch requirements.

Strategic value: Lux drives adoption of other Punt Labs tools via two mechanisms. First, Lux surfaces those tools in its UI — the beads browser, biff presence panels, vox controls, and quarry search all render as Lux extensions; users who see them discover the underlying tools. Second, the install script bundles the full Punt Labs CLI suite. Validation target: 10% of Lux users install one additional Punt Labs tool within 6 months.

Q16: What are the key metrics for success?

1. **Daily active display sessions** — target: 100 within 3 months of launch.
2. **Community extensions published** — target: 50 within 6 months.
3. **Extension install rate** — target: average user installs 3+ community extensions.
4. **Agent-authored extension success rate** — target: 80% of authoring attempts produce a working extension on first try.
5. **Time to new capability** — target: median under 5 minutes from "I need X" to "X is rendering."

Decision threshold: if daily active sessions are below 30 after 6 months, revisit whether the self-extending display server framing resonates or whether Lux should remain a simpler display surface with a fixed vocabulary.

Q17: Why now? What has changed?

Four things converged. (1) The most powerful AI coding agents — Claude Code, Codex — are terminal-hosted with millions of users [CITATION NEEDED], all producing output invisible beyond scrollback. (2) Vendors are responding with web dashboards that require running the entire agent on vendor compute, creating a gap for local-first visual output. (3) LLMs can write UI extensions on demand — the authoring loop that was impossible two years ago is now routine (proven on Postern/Pharo). (4) Lux proved the display surface has value and the protocol-as-API design works.

Q18: What lessons did Lux v1 teach?

Five lessons from daily use:

1. The display has value. The beads browser changed daily workflow.
2. Bidirectional matters. Passive display is not enough; the display must drive agent behavior (via events, menu clicks, interaction messages).
3. Fixed vocabularies are limiting. Element kinds are both too many (most unused) and too few (missing what users actually need).
4. The hook architecture works. PostToolUse auto-refresh makes the display feel live without explicit agent coordination.
5. Multi-agent display is real. Per-project frames, shared ownership, and orphan persistence solve the N-to-1 problem at the protocol level.

Q19: How does the LLM extension authoring loop work?

An agent calls `/api/extensions/author` with a description. The in-display agent (Anthropic Python SDK) generates the extension: a manifest declaring the element kind, schema, dependencies, and rendering mode, plus a renderer function. It runs tests. A consent dialog shows the generated code. The user approves. The extension installs to `~/.lux/extensions/` and is available immediately.

The in-display agent can screenshot the rendered result and iterate — "the table is misaligned, fix the column widths" — using the same technique proven on Postern/Pharo. This visual feedback loop is what makes rapid authoring realistic.

Q20: What resources does this require?

Lux is built by one engineer with AI agent assistance (Claude Code driving implementation, review, and testing — the same model used for Lux's 651 tests and the Smalltalk SDK's 811 tests). Phases 1–3 (extension system, service API, reference extensions) are the priority; other Punt Labs tools (biff, vox, quarry) slow to maintenance-only during that work. Phases 4–6 are sequenced after phases 1–3 ship and external demand is validated.

Q21: Pre-mortem: it is 12 months from now and the self-extending display concept has failed. What happened?

Five plausible failure scenarios:

1. **Anthropic ships built-in visual output in Claude Code.** A first-party surface, even a basic one, eliminates Lux's install-friction advantage. If it is also extensible, Lux's core differentiation collapses.
2. **External developers do not care about agent visibility.** The Punt Labs team uses the beads browser daily, but this may be a niche workflow. Fewer than half of interviewed developers describing visibility as a top-3 pain point means the market is smaller than projected.
3. **Extension authoring success rate is below 50%.** Agents failing more often than they succeed turns the "five-minute" promise into "thirty minutes of debugging," and users give up.
4. **Community registry never bootstraps.** Below 100 active users, the registry has too few extensions to attract users and too few users to attract authors. The flywheel never spins.
5. **Three rendering modes fragment the UX.** Window-mode extensions feel disconnected; texture-mode interactivity is too limited. Users perceive Lux as inconsistent rather than flexible.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	The current product proved the display has value for Punt Labs. Unproven: external developer adoption, whether the extension model resonates beyond power users, and whether the "visual surface for terminal agents without giving up local compute" positioning lands with developers who currently accept text-only output.
Usability	Medium	Install and basic display are proven (one curl command, auto-open). Extension consumption is plausible but untested with external users. Extension authoring — the headline differentiator — requires a multi-step consent flow (generate, review, approve, iterate via screenshot) not yet tested outside Punt Labs.
Feasibility	Medium	Core rendering and IPC are proven. Extension loader is a well-understood pattern. New risks: three rendering modes composing coherently (only ImGui exists currently), in-display agent SDK integration, community registry bootstrapping, and remote TCP/TLS. No single risk is High, but the aggregate — 12+ HTTP endpoints, a tool-use loop, five permission modes, three render contexts — is substantial new work. Outside view: single-engineer 6-phase rewrites with new extension systems typically underestimate scope. Mitigated by phased delivery with a defined cut line after phase 3.
Viability	Medium	Open-source with no direct revenue. Strategic value depends on ecosystem pull for other Punt Labs tools. If Lux does not drive adoption of biff/vox/quarry/beadle, the investment is a public good without business return. Acceptable if treated as infrastructure, not a profit center.

Feature Appendix

Defines the scope boundary for the self-extending display concept. Every feature is categorized to prevent scope creep and force prioritization upfront.

Must Do

Essential for the concept to be distinguishable from current Lux.

- **F1.** **Extension registry and loader** — runtime dispatch table replacing hardcoded element kinds; scan, validate, register, crash-isolate
- **F2.** **Extension manifest format** — declared element kind, schema, dependencies, rendering mode (imgui/texture/window), metadata
- **F3.** **Starter pack migration** — existing element kinds refactored as bundled extensions, not core code
- **F4.** **HTTP service API** — ollama-shaped endpoints for show, update, events, extensions, capabilities; MCP becomes thin adapter
- **F5.** **Auto-documenting `/help` endpoint** — runtime truth from live registry: installed kinds, services, conventions, permission mode
- **F6.** **Extension CLI** — install, list, and remove extensions via `lux ext`; user-local persistence at `~/.lux/extensions/`
- **F7.** **Safety wrapper** — exception isolation per extension render call; crash shows error placeholder, core keeps rendering
- **F8.** **Consent model for extension install** — show source, user approves; same trust model as render_function

Should Do

Not launch-blocking. Fast follow-up candidates.

- **F9.** **In-display agent** — Anthropic SDK, LuxExchange tool-use loop, leaf/composite commands, five-mode permission policy
- **F10.** **LLM extension authoring endpoint** — `/api/extensions/author` driven by in-display agent; screenshot-iterate loop
- **F11.** **Community registry** — GitHub-hosted extension store with publish and search; ratings, download counts, verification badges (lint, CI, tests)
- **F12.** **Reference extensions** — matplotlib plot, terminal (pyte), plotly snapshot, sankey diagram, confusion matrix
- **F13.** **Texture rendering mode** — extensions produce pixel buffers displayed as ImGui images; integrates matplotlib and PIL
- **F14.** **Window rendering mode** — extensions own native OS windows; Lux tracks lifecycle and coordinates focus
- **F15.** **Remote transport** — TCP with TLS; multi-machine agent sessions connecting to one display
- **F16.** **Screenshot API** — capture display state as image for LLM-driven GUI iteration

Won't Do

Explicitly excluded to clarify scope.

- **F17.** **Web-based rendering** — Lux renders natively via ImGui/SDL2 on the developer's GPU. HTML, Electron, and webviews are out of scope — web rendering paths lead to vendor-compute dependency, which is the problem Lux exists to avoid
- **F18.** **GUI framework** — Lux hosts extensions; it does not define a widget class hierarchy. No Lux-specific widget SDK
- **F19.** **Local LLM inference** — the in-display agent calls Anthropic's API; Lux does not host models locally
- **F20.** **OS-level window manager** — window-mode extensions own native OS windows; Lux coordinates lifecycle and focus but does not tile, composite, or manage window placement
- **F21.** **Mobile or tablet** — desktop only (macOS, Linux)

References

[1] Anthropic. *Claude Code*. Agentic coding tool for the terminal. 2025. URL: https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview.

[2] Omar Cornut. *Dear ImGui*. Immediate-mode GUI library for C++. 2025. URL: https://github.com/ocornut/imgui.

[3] Steve Yegge. *The Death of the Junior Developer*. On the difficulty of coordinating multiple AI agents. 2025. URL: https://sourcegraph.com/blog/the-death-of-the-junior-developer.

[4] Ollama. *Ollama*. Local LLM service with HTTP API. 2025. URL: https://ollama.com.

[5] Punt Labs. *Postern: Drive a Live Pharo Image from Any Coding Agent*. HTTP eval server for Pharo with self-documenting /help endpoint. 2026. URL: https://github.com/punt-labs/postern.

[6] Punt Labs. *Claude Agent SDK for Smalltalk*. Five Pharo products for building Claude agents inside a live Smalltalk image. 2026. URL: https://github.com/punt-labs/claude-agent-sdk-smalltalk.